2025-2026

Welcome to the 13th Lab session of CO3519 module (Semester 02)

For Two Hours Lab Session:

#	Task	Time (m)	Guidance
A	Please perform the **Task01** given on **Page 1**.	55	This task is the implementation of CNN as explained in detailed in the today's lecture. Please start from Page 2 to Page 5 to perform Task01. All the instructions are provided with each task to be done.
	Break	05	
B	Please perform the **Task02** given on **Page 6**	55	This task is the utilizing the same implementation which is used in the above **Task A**. You just need to modify the code for the CIFAR10 dataset. Also, perform testing using different images with objects to see how your model performs. You need to complete the sub-tasks for binary and multi-classification.

For One Hour Lab Session:

#	Task	Time (m)	Guidance
1	Please perform the Task **A** from above two hours lab session with the given instructions	50	Follow the related guide in Task **A**.

I recommend downloading this PDF file so that you can view each line of code clearly.

Understanding Convolutional Neural Network and its Implementation for Classification Problem

Important Note: Every student advised to carefully review each line and develop a clear understanding of the concepts. As we progress to more advanced and technical sessions in the coming weeks, it is essential to have a strong grasp of these foundational concepts. This will also support your preparation for the second assessment.

Task01: Implementation of CNNs for DIGIT classification using MNIST dataset

In the previous week, we implemented an **ANN** solved for **XOR** problem. This week, we will dive into the understanding of Convolutional Neural Network CNN as these are the core and main part of Artificial Intelligence. We consider DIGITS dataset for its training on CNN to classify digits.

About DIGITS Dataset: This dataset is the collection of handwritten numerical digits commonly used for image classification and machine learning experiments. The most well-known variant is the MNIST dataset, which consists of 60,000 training images and 10,000 test images of grayscale handwritten digits (0-9) with a resolution of 28x28 pixels. Each image represents a single-digit number, and the dataset is widely used as a benchmark for evaluating machine learning models, particularly in deep learning and convolutional neural networks (CNNs). The dataset is essential in the field of computer vision and serves as an introductory problem for classification tasks due to its simplicity, large size, and well-labelled format.

First, we need to import the essential libraries. Essential libraries:
TensorFlow and Keras for building and training CNN models while NumPy for handling numerical computations and Matplotlib will visualize the results.

Please make sure you have installed these libraries or use Google Collab to avoid issues.

```
[1]
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
```

Here, we load the MNIST dataset and normalize pixel values between 0 and 1 to speed up training and improve convergence.

```
[2]
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
# (x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

# Reshape dataset to match model's expected input shape (28,28,1)
x_train = x_train.reshape(-1, 28, 28, 1).astype("float32") / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype("float32") / 255.0

```

Next, as we imported the essential libraries and then downloaded the dataset, lets design/define the architecture of the CNN model.

First Convolutional Layer:

Conv2D (32, (3,3), activation='relu'): This layer extracts 32 different patterns from the input using 3x3 filters. The ReLU activation introduces non-linearity, making it easier to learn complex patterns.
MaxPooling2D ((2,2)): Downsamples the feature maps by selecting the most important features, reducing computation and preventing overfitting.

Second Convolutional Layer:

Conv2D(64, (3,3), activation='relu'): Uses 64 filters to extract more refined patterns from previous feature maps.
MaxPooling2D((2,2)): Further reduces dimensionality while keeping key features.

Flatten Layer:

Flatten(): Converts 2D feature maps into a 1D vector, making it compatible with dense layers.

Fully Connected Layers:
Dense(64, activation='relu'): A hidden layer with 64 neurons to learn high-level representations.
Dense(10, activation='softmax'): Output layer with 10 neurons (one per digit), using softmax activation for probability distribution.

Followed by the compilation of the model, where

Adam Optimizer: An adaptive learning rate method that helps achieve faster convergence.
Sparse Categorical Crossentropy: A loss function used for multi-class classification with integer labels.
Accuracy Metric: Measures how often the model correctly classifies the digits.

```
[3]
model = keras.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Let's start training. We train the CNN for 5 epochs, meaning the model sees the entire dataset 5 times. The validation data helps monitor performance and detect overfitting.

```
[4]
history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

We visualize the training and validation accuracy across epochs. A large gap between them could indicate overfitting. Although, the graph/curve will not be too smooth as there are only 5 epochs applied to train the model.

```
[5]  plt.plot(history.history['accuracy'], label='accuracy')
     plt.plot(history.history['val_accuracy'], label='val_accuracy')
     plt.xlabel('Epochs')
     plt.ylabel('Accuracy')
     plt.legend()
     plt.show()
```

If you want to see which images/digits are wrongly classified, you can run the following snippet.

```
[6]  import random
     predictions = model.predict(x_test)
     misclassified_indices = [i for i in range(len(y_test)) if np.argmax(predictions[i]) != y_test[i]]
     random_samples = random.sample(misclassified_indices, 5)

     for i in random_samples:
         plt.imshow(x_test[i], cmap='gray')
         plt.title(f"True: {y_test[i]}, Predicted: {np.argmax(predictions[i])}")
         plt.show()
```

Single Image Test.

I have added few images in this week folder, which you can test such as (**1.jpeg**, **5.jpg**) images in test folder.

Now, as we did the single image in the previous semester while training the model and then performing the testing of a single image, now we do the same here. This code is used to load, process, and display an image classified. First, it loads a grayscale image from the specified file path using OpenCV (cv2.imread). If the image is successfully loaded, it proceeds with two tasks: displaying the original image and then processing it for digit recognition. The image is resized to 28x28 pixels to match the dimensions expected by a pre-trained model (here, we trained MNIST dataset). Additionally, the colors are inverted (since MNIST images typically have white digits on a black background). The image is normalized and reshaped into the required format for model input. Afterward, it uses a deep learning model (model.predict) to make a prediction about the digit in the image. The result (predicted label) is then displayed alongside the processed image. Both the original and the processed images are displayed side by side using matplotlib's subplot function, allowing for visual comparison.

So, do it, follow this code:

First, mount the drive.

```
[8]  from google.colab import drive
     drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; t
```

```python
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Load the image from file
image_path = "/content/drive/My Drive/5.jpg"
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image not loaded. Check file path.")
else:
    # Display the original and processed images
    plt.figure(figsize=(10, 5))

    # Original Image
    plt.subplot(1, 2, 1)
    plt.imshow(image, cmap='gray')
    plt.title("Original Image")
    plt.axis('off')

    # Resize image to match MNIST dimensions
    image_resized = cv2.resize(image, (28, 28))

    # Invert colors if needed (MNIST is white digits on black background)
    image_resized = 255 - image_resized

    # Normalize and reshape the image
    image_resized = image_resized.astype("float32") / 255.0
    image_resized = image_resized.reshape(1, 28, 28, 1)  # Ensure correct input shape

    prediction = model.predict(image_resized)
    predicted_label = np.argmax(prediction)

    # Processed Image
    plt.subplot(1, 2, 2)
    plt.imshow(image_resized.reshape(28, 28), cmap='gray')
    plt.title(f"Processed Image\nPredicted Label: {predicted_label}")
    plt.axis('off')
    plt.show()
```

Task02: Please use the same code and implement the CNN for CIFAR10 Dataset.

After implementing CNN for CIFAR10, please read "LeCun Y, Bengio Y. Convolutional networks for images, speech, and time series. The handbook of brain theory and neural networks. 1995 Apr;3361(10):1995." and note the key details.

About CIFAR Dataset: The CIFAR (Canadian Institute for Advanced Research) dataset is a widely used dataset in computer vision and deep learning, consisting of two versions: CIFAR-10 and CIFAR-100. CIFAR-10 contains 60,000 colour images of size 32×32 pixels, categorized into 10 classes, including airplanes, cars, birds, cats, and more, with 6,000 images per class. Next, it has Training images: 50,000 (5,000 per class), Testing images: 10,000 (1,000 per class), Total images: 60,000.

Don't use CIFAR-100 but understand it. It is a more complex version with 100 categories, each containing 600 images, making classification more challenging. The dataset is commonly used for training and evaluating image classification models, particularly convolutional neural networks (CNNs). Due to its relatively small image size and diverse object categories, CIFAR serves as an excellent dataset for experimenting with deep learning architectures and improving model generalization in real-world vision tasks.

Sub-part of Task02:

Please implement custom CNN model for **Cat** and **Dog** (Binary classification). You can consider the training and test set we used for training in the previous semester. Compute the confusion matrix and perform the single image test. You can examine the results and improve by changing/updating the parameters.

After doing this, you can add a third class of your choice (maybe Airplane, Bicycle, building, etc.) to perform multi-class classification. You can search for the images/data on google, for instance, with querying '*airplane images dataset'* etc. (Commercial Aircraft Dataset).